

CampusOS AI

A private, offline study assistant that turns a student's own course material into a voice-driven learning loop.

Snapdragon® AI Lab Build & Present Challenge · Solution Submission Round · Target platform: Snapdragon-powered HP PCs

1. The problem

Indian college students preparing for exams face four problems at once, and the tools available to them solve at most two.

- Their material is specific and theirs — the syllabus, the professor's slides, last year's question paper. A general chatbot has never seen any of it and answers from something else.
- They cannot verify what they are told. A fluent wrong answer is indistinguishable from a right one when you are the person still learning the topic. This is exactly the population least equipped to catch a hallucination.
- The explanation language is not the material language. Notes are in English; understanding happens in Hindi or a Hindi-English mix. Students perform that translation in their heads, badly, under time pressure.
- They do not know what they do not know. Re-reading feels like studying. It is the least effective way to find gaps, and it is the default because nothing tells them where the gaps are.

Connectivity adds a fifth. Hostel and campus Wi-Fi is unreliable, and cloud AI stops working during exactly the late-night sessions when it is needed most.

2. What CampusOS AI does

A student drops their own PDFs into the app. They ask a question out loud. They get an explanation drawn only from their material, in English, Hindi or Hinglish, with page citations they can click and check. Then the app quizzes them on it and tells them which topic they are actually weak on — and links that topic straight back into a grounded explanation of it.

Everything runs on the laptop. No account, no API key, no cloud service, and after first-run model download, no network at all.

The loop is the product. Question-answering is one step inside it, not the whole thing.

3. What makes it different

Property	How it is enforced
It refuses when it does not know	A relevance gate rejects out-of-corpus questions before the language model is ever called. The model cannot talk its way past it.
Every citation is real	Citations are validated against the chunks actually supplied to the model. A page reference the model invented is dropped before the student sees it.
The loop ends in a quiz	Each question carries the source chunk it was generated from, so its explanation is grounded by construction and the weak topic it names is traceable to a page.
It speaks Hinglish	The explanation is re-expressed in the register students actually think in, with citation markers preserved intact.
Offline is proven, not promised	An automated test blocks every non-loopback socket and runs the full pipeline anyway.

The strongest technical claim here is a negative one: the system is built so that it cannot cite a page it was not shown. That is enforced in code and covered by tests, not requested in a prompt.

4. Architecture

A Python backend bound to 127.0.0.1 serves a static frontend. All inference is local. The organising principle is that every AI runtime is reached through an interface that has both an x86-64 implementation and a Snapdragon implementation, so porting is configuration rather than engineering.

Workload	Interface	Development host (x64)	Target: Snapdragon-powered HP PC
Language model	OpenAI-compatible HTTP	llama.cpp server (CPU)	GenieX / llama.cpp ARM64 (NPU/GPU)
Embeddings	ONNX Runtime session	CPUExecutionProvider	QNNExecutionProvider (QnnHtp.dll)
Speech-to-text	ONNX Runtime session	faster-whisper (CPU)	Qualcomm AI Hub Whisper via QNN EP
Retrieval	NumPy	CPU	CPU — sub-millisecond at this scale

Because the language model runs out-of-process behind an OpenAI-compatible endpoint, swapping it for GenieX on a Snapdragon machine is an environment variable, not a code change. The provider list for ONNX Runtime is selected at startup by platform detection, with QNN first and CPU immediately after it in the same list, so an unsupported graph degrades gracefully instead of crashing.

5. Optimisation for Snapdragon-powered HP PCs

This project is designed and built to be optimised for Snapdragon-powered HP PCs. The work done toward that is concrete:

- Platform detection and QNN execution-provider selection are implemented in the shipped code, not described in a document. The active provider is reported by the `/api/health` endpoint and recorded in every benchmark run.
- The model format was chosen for portability: the same GGUF weights run under llama.cpp on x64 and under GenieX on Snapdragon, with no conversion step.
- The benchmark harness runs unmodified on ARM64, so the numbers from both machines are directly comparable rather than merely similar.
- A full ARM64 setup guide ships in the repository (`scripts/setup_snapdragon.md`), covering the ARM64 Python requirement, the `onnxruntime-qnn` package swap, the `QnnHtp.dll` backend path, and how to verify that QNN actually loaded.
- No application code changes are required to move between the two platforms.

A statement we consider more important than a favourable number

No NPU execution has been measured in this project, because the development machine has no NPU. Per ONNX Runtime's documentation, the win-x64 build supports QNN quantization and ahead-of-time compilation only — not inference. Every NPU statement in this submission is therefore a documented deployment path with a citation, never a measured result.

The Snapdragon column in our benchmark table is deliberately left empty and labelled "requires hardware". We would rather present an honest gap that a judge can verify than a projected figure that would not survive scrutiny.

6. Measured results

Measured on the development machine: Intel x86-64, 4 physical cores, 8.2 GB RAM, Windows 11, no NPU. CPU execution throughout.

Metric	This machine (x64, CPU)	Snapdragon X (NPU)
Ingest throughput	11.9 pages/s	requires hardware
Query embedding latency	6.5 ms	requires hardware
Retrieval latency	7.7–10.1 ms	requires hardware
Backend resident memory	508–756 MB	requires hardware
Language model resident memory	~2.0 GB (3B, Q4_K_M)	requires hardware
Outbound network connections during a full cycle	0 (asserted by test)	requires hardware

Answer latency is reported in the repository rather than summarised here, because it is dominated by available memory on an 8 GB machine and varies with what else is running. Streaming is used so that text appears as it is generated rather than after a wait. We regard publishing a latency figure without its memory conditions as misleading, so both are recorded together.

7. Privacy

- Documents, vectors, transcripts and quiz history stay in a local application-data directory.
- No telemetry, no analytics, no crash reporting — none exists anywhere in the codebase.
- Audio is transcribed and the buffer discarded. It is never written to the application data directory.
- Deleting a document is a hard delete: SQLite rows are removed, the vector array is rebuilt without them, and a test asserts the vector count returns to zero.
- The only network event in the product’s life is the first-run model download, performed by a script the user runs deliberately. No document ever leaves the machine.

8. Verifying these claims

Every claim in this document corresponds to something a judge can run from the repository.

Command	What it proves
python tests\test_pipeline.py	Ingest, retrieval, the refusal gate and hard deletion
python tests\test_egress.py	Blocks all non-loopback sockets, runs the pipeline, asserts zero outbound connections
python tests\test_asr.py	Local speech-to-text on synthesized audio
python tests\test_frontend.py	Frontend wiring and the keyboard-accessibility floor
python benchmarks\run.py	Regenerates the results table, including the active execution provider
python eval\run_eval.py	Groundedness and out-of-corpus refusal against a fixed evaluation set

9. Accessibility and deployment

- Runs within an 8 GB RAM floor on x64 or ARM64 Windows 11.
- Started with a single command: `python -m backend`.

- Fully keyboard-operable — voice input is never the only path. Transcripts are always shown as text, so audio is never the sole channel.
- No accounts, no login, no build step for the frontend, and therefore no toolchain to fail on the target machine.

Repository: <https://github.com/abhayjeet5465/campusOs>

Models are downloaded from their original publishers at first run and are not redistributed. Third-party licence notices, including the Llama 3.2 Community Licence attribution, are in LICENSE. Built with Llama.